

pHostInfo, uPokeDB, and uQueryDB

Host info and MOOSDB poke/query utilities.

Included MIT MOOS-IvP PDF sources:

- app_phostinfo.pdf
- app_upokedb.pdf
- app_uquerydb.pdf

pHostInfo: Detecting and Sharing Host Info

June 2018

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1	Overview	1
2	Configuration Parameters for pHostInfo	2
3	Publications and Subscriptions for pHostInfo	3
3.1	Variables Published by pHostInfo	3
3.2	Variables Subscribed for by pHostInfo	4
3.3	Command Line Usage of pHostInfo	4
4	Usage Scenarios for the pHostInfo Utility	4
4.1	Handling Multiple IP Addresses	4
5	A Peek Under the Hood	5
5.1	Temporary Files	5
5.2	Possible Gotchas	5

1 Overview

The **pHostInfo** application is a tool with a simple objective - determine the IP address of the machine on which it is running and post it to the MOOSDB. Although this information is available in a number of ways to a user at the keyboard, it may not be readily available for reasoning about within a MOOS community. Often, from an application's perspective, the host name is simply known and configured as *localhost*. This is fine for most purposes, but in situations where a user is on a machine where the IP address changes frequently, and the user is launching MOOS processes that talk to other machines, it may be very convenient to auto-determine the prevailing IP address and publish it to the MOOSDB. The typical usage scenario for **pHostInfo** is shown in Figure 1.

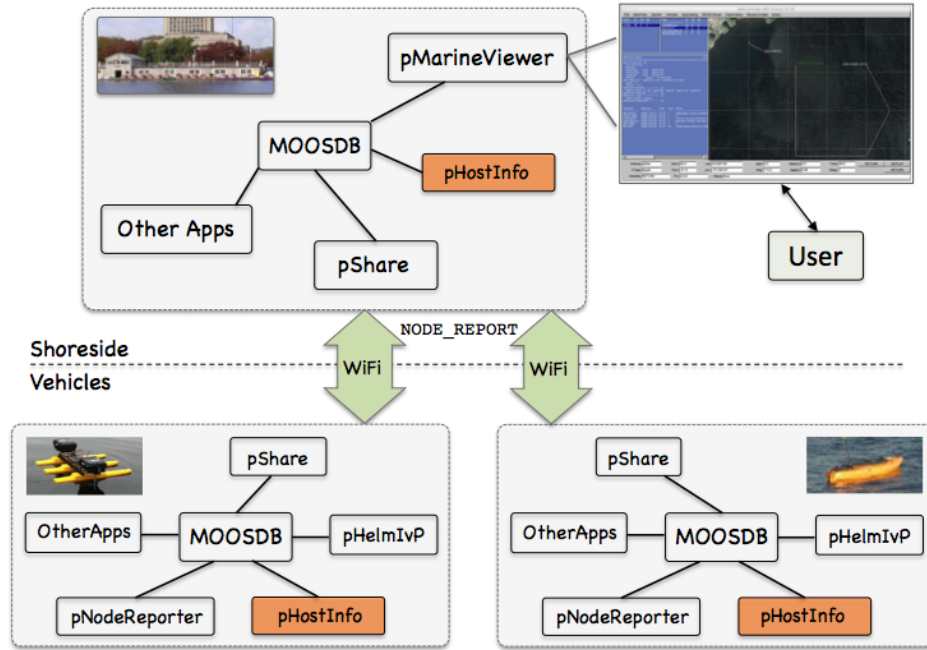


Figure 1: **Typical pHostInfo Topology:** A shoreside or topside community is receiving information from several deployed vehicles, in the form of node reports. The node reports contain time-stamped updated vehicle positions, from which the speed and distance measurements are derived and posted to the shoreside MOOSDB.

There are two scenarios where this is currently envisioned to be useful. The first is when the fielded vehicles are on the network with their IP addresses set via DHCP. For example if on the network via a cellular phone connection. The second is if the "vehicle" is a simulated vehicle running on a user's laptop also with its IP address set via DHCP. In both cases there may be another MOOS community with a known IP address, e.g., a shoreside community, to which the local vehicle wishes to inform it of its current IP address. This simple process however does not get involved in any activity regarding the communication to other MOOS communities, but simply tries to determine and post, the IP address, e.g., `PHI_HOST_IP="192.168.0.1"` for other applications to do as they see fit.

2 Configuration Parameters for pHostInfo

The `pHostInfo` application may be configured with a configuration block within a MOOS mission file, typically with a `.moos` file suffix. The following parameters are defined for `pHostInfo`.

Listing 2.1: Configuration Parameters for pHostInfo.

<code>temp_file_dir:</code>	Directory where temporary files are written. Default is <code>"~/."</code> .
<code>default_hostip:</code>	IP address used if no IP address can otherwise be determined.
<code>default_hostip_force:</code>	This IP address will override any IP address from an auto-discovered network interface. Useful for debugging.

`prefer_interface:` If multiple interfaces have a valid IP address then a preference can be specified. Valid options are `wlan0`, `wifi`, `eth0`, `eth1`, `usb0`, `usb1`, `usb2`.

An Example MOOS Configuration Block

An example MOOS configuration block may be obtained from the command line with the following:

```
$ pHostInfo --example or -e
```

Listing 2.2: Example configuration of the pHostInfo application.

```
1 =====
2 pHostInfo Example MOOS Configuration
3 =====
4
5 ProcessConfig = pHostInfo
6 {
7     AppTick    = 4
8     CommsTick  = 4
9
10    temp_file_dir = ./
11    default_hostip = 192.168.0.55    // default is "localhost"
12
13    default_hostip_force = 192.168.0.99
14 }
```

3 Publications and Subscriptions for pHostInfo

The interface for `pHostInfo`, in terms of publications and subscriptions, is described below. This same information may also be obtained from the terminal with:

```
$ pHostInfo --interface or -i
```

3.1 Variables Published by pHostInfo

The primary output of `pHostInfo` to the MOOSDB are the following five `PHI_*` variables. Once these variables are published, `pHostInfo` does not publish them again unless requested by receiving mail `HOST_INFO_REQUEST`. Thus `pHostInfo` is mostly idle once the below five variables are posted.

- `APPCAST`: Contains an appcast report identical to the terminal output. Appcasts are posted only after an appcast request is received from an appcast viewing utility.
- `PHI_HOST_IP`: The single best guess of the Host's IP address.
- `PHI_HOST_IP_ALL`: A comma-separated list of IP addresses if multiple addresses detected.
- `PHI_HOST_IP_VERBOSE`: A comma-separated list of IP addresses, with source information, if multiple addresses detected.
- `PHI_HOST_PORT_DB`: The port number of the MOOSDB for this community.

- **PHI_HOST_PORT_INFO**: A comma-separated list of parameter-value pairs describing all relevant aspects of the host.

3.2 Variables Subscribed for by pHostInfo

The **pHostInfo** application subscribes to the following MOOS variable:

- **APPCAST_REQ**: A request to generate and post a new apppcast report, with reporting criteria, and expiration. Section ??.
- **HOST_INFO_REQUEST**: A request to re-determine and re-post the platform's host information.
- **PSHARE_INPUT_SUMMARY**: The input routes used by **pShare** for listening for incoming messages from other MOOSDBs.

3.3 Command Line Usage of pHostInfo

The **pHostInfo** application is typically launched with **pAntler**, along with a group of other modules. However, it may be launched separately from the command line. The command line options may be shown by typing:

```
$ pHostInfo --help or -h
```

Listing 3.3: Command line usage for the pHostInfo tool.

```

1  Usage: pHostInfo file.moos [OPTIONS]
2
3  Options:
4    --alias=<ProcessName>
5        Launch pHostInfo with the given process
6        name rather than pHostInfo.
7    --example, -e
8        Display example MOOS configuration block
9    --help, -h
10       Display this help message.
11    --HOSTIP=<HostIP>
12       Force the use of the given IP address as the reported IP
13       address ignoring any other auto-discovered IP address.
14    --interface, -i
15       Display MOOS publications and subscriptions.
16    --version, -v
17       Display the release version of pHostInfo.
18
19  Note: If argv[2] is not of one of the above formats
20       this will be interpreted as a run alias. This
21       is to support pAntler launching conventions.
```

4 Usage Scenarios for the pHostInfo Utility

4.1 Handling Multiple IP Addresses

It is possible that a machine has more than one valid IP address at any given time, e.g., if its ethernet cable is plugged in, and it has a wireless connection. In this case, **pHostInfo** will make a guess that

the ethernet connection takes precedent, and it will report this in the variable `PHI_HOST_IP`. The full set of IP addresses can be found in the other postings. For example it may not be uncommon to see something like the following three postings at one time:

```
PHI_HOST_IP          = 118.10.24.23
PHI_HOST_IP_ALL      = 118.10.24.23,169.224.126.40
PHI_HOST_IP_VERBOSE = OSX_ETHERNET2=118.10.24.23,OSX_AIRPORT=169.224.126.40
```

5 A Peek Under the Hood

The `pHostInfo` application currently only works for GNU/Linux and Apple OS X. It determines the IP information by making a system call within C++. A system call when generated will act as if the argument were typed on the command line. In this case the system call is generated and the output is redirected to a file. In a second step, `pHostInfo` then tries to read the IP address information from those files.

In GNU/Linux, the system call is based on the `ifconfig` command. In OS X, the system call is based on the `networksetup` command. Rather than determining in `pHostInfo` whether the user is running in a GNU/Linux or OS X environment, the system calls for both are invoked. Presumably a system call on a command not found in the user's shell path will not generate something that is confusable with a valid IP address.

5.1 Temporary Files

The temporary files are written to the user's home directory by default. This may be changed with the `temp_file_dir` configuration parameter, for example, `temp_file_dir=/tmp`. The set of temporary files are put into a folder named `.phostinfo/`. The set of temporary files may look like:

```
$ cd ~/.phostinfo
$ ls -a .ipinfo*
.ipinfo_linux_ethernet.txt    .ipinfo_osx_airport.txt    .ipinfo_osx_ethernet2.txt
.ipinfo_osx_wifi.txt         .ipinfo_linux_wifi.txt    .ipinfo_osx_ethernet1.txt
.ipinfo_osx_ethernet.txt
```

Some of these files may be empty, or some may contain error output if one of the system commands was not found, or was given an improper argument. The `pHostInfo` app will try to parse all of them to find a valid IP address. If more than one IP address is found, then this handled in the manner described previously in Section 4.1.

5.2 Possible Gotchas

The system calls invoked by `pHostInfo` need to be in the users shell path. A typical user default environment would have these in their shell path anyway, but it may be worth checking if things aren't working properly. Below is a list of commands that are run under the hood, and their probable locations on your system.

For Linux:

/sbin/ifconfig

/bin/grep

/usr/bin/cut

/usr/bin/awk

/usr/bin/print

For OS X:

/usr/sbin/networksetup

uPokeDB: Poking the MOOSDB from the Command Line

June 2018

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1	Overview	1
2	Command-line Arguments of uPokeDB	2
3	MOOS Poke Macro Expansion	2
4	Providing the ServerHost and ServerPort on the Command Line	3
5	Session Output from uPokeDB	3
6	Publications and Subscriptions for uPokeDB	4

1 Overview

The **uPokeDB** application is a lightweight process that runs without any user interaction for writing to (poking) a running **MOOSDB** with one or more variable-value pairs. It is run from a console window with no GUI. For example, the alpha example mission is normally kicked off by hitting the **DEPLOY** button. The same could be accomplished from the terminal with:

```
$ uPokeDB alpha.moos DEPLOY=true, MOOS_MANUAL_OVERRIDE=false
```

After accepting variable-value pairs from the command line, **uPokeDB** connects to the **MOOSDB**, displays the variable values prior to poking, performs the poke, displays the variable values after poking, and then disconnects from the **MOOSDB** and terminates. It also accepts a **.moos** file as a command line argument to grab the IP and port information to find the **MOOSDB** for connecting. Other than that, it does not read a **uPokeDB** configuration block from the **.moos** file.

Other Methods for Poking a MOOSDB

There are few other **MOOS** applications capable of poking a **MOOSDB**. The **uMS** (**MOOS Scope**) is an application for both monitoring and poking a **MOOSDB**. It is substantially more feature rich than **uPokeDB**, and depends on the **FLTK** library. The **iRemote** application can poke the **MOOSDB** by using the **CustomKey** parameter, but is limited to the free unmapped keyboard keys, and is good when used with some planning ahead. The latest versions of **uMS** and **iRemote** are maintained on the Oxford **MOOS** website. The **uTermCommand** application is a tool primarily for poking the **MOOSDB** with a pre-defined list of variable-value pairs configured in its **.moos** file configuration block. The user initiates each poke by entering a keyword at a terminal window. Unlike **iRemote**

it associates a variable-value pair with a key *word* rather than a keyboard key. The `uTimerScript` application is another tool for poking the MOOSDB with a pre-defined list of variable-value pairs configured in its `.moos` file configuration block. Unlike `uTermCommand`, `uTimerScript` will poke the MOOSDB without requiring further user action, but instead executes its pokes based on a timed script. The `uMOOSPoke` application, written by Matt Grund, is similar in intent to `uPokeDB` in that it accepts a command line variable-value pair. `uPokeDB` has a few additional features described below, namely multiple command-line pokes, accepting a `.moos` file on the command-line, and a MOOSDB summary prior and after the poke.

2 Command-line Arguments of `uPokeDB`

The command-line invocation of `uPokeDB` accepts two types of arguments - a `.moos` file, and one or more variable-value pairs. The former is optional, and if left unspecified, will infer that the machine and port number to find a running `MOOSDB` process is `localhost` and port `9000`. The `uPokeDB` process does not otherwise look for a `uPokeDB` configuration block in this file. The variable-value pairs are delimited by the '=' character as in the following example:

```
$ uPokeDB alpha.moos FOO=bar TEMP=98.6 MOTTO="such is life" TEMP_STRING:=98.6
```

Since white-space characters on a command line delineate arguments, the use of double-quotes must be used if one wants to refer to a string value with white-space as in the third variable-value pair above. The value *type* in the variable-value pair is assumed to be a double if the value is numerical, and assumed to be a string type otherwise. If one really wants to poke with a string type that happens to be numerical, i.e., the string "98.6", then the ":@" separator must be used as in the last argument in the example above. If `uPokeDB` is invoked with a variable type different than that already associated with a variable in the MOOSDB, the attempted poke simply has no effect.

3 MOOS Poke Macro Expansion

The `uPokeDB` utility supports macro expansion for timestamps. This may be used to generate a proxy posting from another application that uses timestamps as part of it posting. The macro for timestamps is `@MOOSTIME`. This will expand to the value returned by the MOOS function call `MOOSTime()`. This function call is implemented to return UTC time. This following is an example:

```
$ uPokeDB file.moos FOOBAR=color=red,temp=blue,timestamp=@MOOSTIME
```

The above poke would result in a posting similar to:

```
FOOBAR = color=red,temp=blue,timestamp=10376674605.24
```

As with other pokes, if the macro is part of a posting of type double, the timestamp is treated as a double. The posting

```
$ uPokeDB file.moos TIME_OF_START=@MOOSTIME
```

would result in the posting of type double for the variable `TIME_OF_START`, assuming it has not been posted previously as a different type.

4 Providing the ServerHost and ServerPort on the Command Line

The specification of a MOOS file on the command line is optional. The only two pieces of information `uPokeDB` needs from this file are (a) the `server_host` IP address, and (b) the `server_port` number of the running MOOSDB to poke. These values can instead be provided on the command line:

```
$ uPokeDB F00=bar --host=18.38.2.158 --port=9000
```

If the `host` or the `port` are not provided on the command line, and a MOOS file is also not provided, the user will be prompted for the two values. Since the most common scenario by convention has the MOOSDB running on the local machine (“localhost”) with port 9000, these are the default values and the user can simply hit the return key.

```
$ uPokeDB F00=bar           // User launches with no server host/port info
$ Enter Server: [localhost] // User accepts default by hitting Return key
$   The server is set to "localhost" // Server host confirmed to be set to "localhost"
$ Enter Port: [9000] 9123    // User overrides the default 9000 port with 9123
$   The port is set to "9123" // Server port confirmed to be set to "9123"
```

5 Session Output from uPokeDB

The output in Listing 1 shows an example session when a running MOOSDB is poked with the following invocation:

```
$ uPokeDB alpha.moos DEPLOY=true RETURN=true
```

Lines 1-16 are standard output of a MOOS application that has successfully connected to a running MOOSDB. Lines 19-23 indicate the value of the variables prior to being poked, along with their source, i.e., the MOOS process responsible for publishing the current value to the MOOSDB, and the time at which it was last written. The time is given in seconds elapsed since the MOOSDB was started. Lines 26-30 show the new state of the poked variables in the MOOSDB after `uPokeDB` has done its thing.

Listing 5.1: An example uPokeDB session output.

```
1 -----
2 |          This is an Asynchronous MOOS Client          |
3 |          c. P. Newman U. Oxford 2001-2012             |
4 -----
5
6 -----MOOS CONNECT-----
7   contacting a MOOS server localhost:9000 - try 00001
8   Contact Made
9   Handshaking as "uPokeDB"..... [OK]
10 -----
```

```

11
12 uPokeDB is Running:
13 +Baseline AppTick    @ 5.0 Hz
14 +Comms is Full Duplex and Asynchronous
15 +Iterate Mode 0 :
16   -Regular iterate and message delivery at 5 Hz
17
18
19 PRIOR to Poking the MOOSDB
20   VarName           (S)ource   (T)ime   VarValue
21   -----
22   DEPLOY             uTimerScript1.92   "true"
23   RETURN             pHelmIvP    1       "false"
24
25
26 AFTER Poking the MOOSDB
27   VarName           (S)ource   (T)ime   VarValue
28   -----
29   DEPLOY             uPokeDB        22.58   "true"
30   RETURN             uPokeDB        22.58   "false"

```

6 Publications and Subscriptions for uPokeDB

Variables published by the uPokeDB application

- USER-DEFINED: The only variables published are those that are poked. These variables are provided on the command line. See Section 2.

Variables subscribed for by the uPokeDB application

- USER-DEFINED: Since uPokeDB provides two reports as described in the above Section 5, it subscribes for the same variables it is asked to poke, so it can generate its before-and-after reports.

uQueryDB: Querying the MOOSDB from the Command Line

June 2018

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
`project-pavlab/appdocs/app_uquerydb`

1 Overview	1
1.1 A Simple Example	2
2 Using the Output Generated by uQueryDB	4
3 Using uQueryDB within a Shell Script	4
4 Command-line Arguments of uQueryDB	5
4.1 Logic Pass Conditions on the Command Line	6
4.2 Logic Fail Conditions on the Command Line	6
4.3 Waiting for Success	6
4.4 Waiting for a Logic Variable to be Published	7
5 Check Variables	7
5.1 Specifying Check Variables on the Command Line	7
5.2 Specifying Check Variables in the Mission File	7
5.3 Check Variable Output and Format	7
6 Configuration Parameters for uQueryDB	8
7 Publications and Subscriptions for uQueryDB	9

1 Overview

The **uQueryDB** utility is a command-line tool for querying a MOOSDB with a logic condition provided on the command line. It finds the MOOSDB via a mission file provided on the command line, or the IP address and port number given on the command line. It will connect to the DB, register for the variables involved in the logic condition and determine if the condition holds. It will then exit with 0 if it holds or 1 otherwise. It will return its value as soon as the app has received mail for all variables involved in the logic condition. Otherwise it will wait for 10 seconds. This can be changed with the `--wait=N` parameter. If a variable in the logic condition is unknown to the MOOSDB, then the whole condition will fail after the wait period.

The motivation for **uQueryDB** is to facilitate automated testing of missions launched from within a shell script, by checking for termination criteria and automatically halting the test mission.

1.1 A Simple Example

As a simple example, try launching the alpha mission in the `s1_alpha` example mission folder. From the command line, launch with:

```
$ cd moos-ivp/ivp/missions/s1_alpha  
$ ./launch.sh
```

Once this simulation has started, deploy the vehicle with the button on the viewer. At this point we can try out the `uQueryDB` utility. In another terminal window try typing:

```
$ cd moos-ivp/ivp/missions/s1_alpha  
$ uQueryDB alpha.moos --condition="DB_UPTIME<130"
```

This will check the value of the MOOS variable `DB_UPTIME` which is the number of seconds that the MOOSDB has been up. The above check will simply report success until after 130 seconds has passed. The report comes in human-readable form in the terminal output:

```

$ uQueryDB alpha.moos --condition="DB_UPTIME<130"
Mission File was provided: alpha.moos
Condition: [DB_UPTIME<130]

----- moos connect -----
contacting a MOOS server localhost:9000 - try 00001
Handshaking as uQueryDB [ok]
DB reports async support is [on]
DB is running on leonardo
Timing skew estimation is [off] (not needed)
-----

uQueryDB is Running:
|-Baseline AppTick @ 5.0 Hz
|--Comms is Full Duplex and Asynchronous
-Iterate Mode 0 :
|-Regular iterate and message delivery at 5 Hz

=====
uQueryDB alpha 0/0(1)
=====
Config:
m_sServerHost: localhost
m_lServErport: 9005
Max Time: n/a
State:
Start Time: 1640112857.63442
Curr Time: 1640112857.83831
Elapsed Time: n/a
Exit Value: 1
Config (pass/fail):
pass_conditions: 1
fail_conditions: 0
Result: fail: DB_UPTIME<13

InfoBuffer: (pass condition vars)
=====
DB_UPTIME 107.35
WFLAG waypoints=3

InfoBuffer: (fail condition vars)
=====
DB_UPTIME 107.35
WFLAG waypoints=3

InfoBuffer: (check vars)
=====
WFLAG waypoints=3

```

Note in the last two lines the current value of the MOOS variable **DB_UPTIME** is reported and the result of the logic condition is declared along with the value returned by the command, `exit(1)`.

2 Using the Output Generated by uQueryDB

In addition to the human-readable output above, the **uQueryDB** utility also generates a return value available on the command line. In line with common GNU/Linux tradition, this utility returns 0 when it is successful and a non-zero value when not successful. In our case **uQueryDB** returns 0 when the logic condition succeeds, and 1 when it does not. As a simple exercise, launch the alpha mission again as in the previous example. After it has again launched, enter the following in another terminal window:

```
$ uQueryDB alpha.moos --condition="DB_UPTIME<130" >& /dev/null
```

By redirecting to `/dev/null` the normal terminal output is suppressed but the result is generated nevertheless, and can be seen by examining the special shell variable `$?` with:

```
$ echo $?  
0
```

The 0 indicates success, i.e., the logic condition succeeded. This approach is especially useful when used inside a shell script, described next.

3 Using uQueryDB within a Shell Script

The motivation for **uQueryDB** is to support the automation of simulation experiments by launching a mission from within a script and exiting the script (and the mission) when a specified condition is met within the simulation environment. By example we return to the alpha mission. The example script below launches the alpha mission and halts the alpha mission after 20 seconds.

```
#!/bin/bash -e

# Part 1: Launch the mission
pAntler alpha.moos >& /dev/null &

# Part 2: After a short wait deploy the vehicle
sleep 5
uPokeDB alpha.moos MOOS_MANUAL_OVERRIDE=false DEPLOY=true
sleep 5

# Part 3: Continually check for the termination criteria
while true; do
    uQueryDB alpha.moos --condition="DB_UPTIME>30"
    if [ "$?" = 0 ]; then
        echo -n "Quitting..."
        killall pAntler
        sleep 2
        echo "Done."
        exit 0
    fi
    sleep 5
    echo "continuing..."
done
```

Of course the above example simply uses *time* as a termination criteria, which could be accomplished by just changing the `sleep` command and terminating. But `uQueryDB` allows the termination to be based on mission related criteria. For example, in the alpha mission, termination can be set to happen after the vehicle traverses its set of waypoints twice, by setting the logic condition with `--condition="CYCLE_INDEX>2"`.

4 Command-line Arguments of uQueryDB

Configuration parameters can be passed to `uQueryDB` from the command line or from the mission (`.moos`) configuration file. Launching it without logic conditions will simply result in `uQueryDB` returning immediately with a value of 0 (success).

The specification of a MOOS file on the command line is optional. The primary two pieces of information `uQueryDB` needs from this file are (a) the `server_host` IP address, and (b) the `server_port` number of the running MOOSDB to poke. These values can instead be provided on the command line:

```
$ uQueryDB --condition="NUM>30" --host=18.38.2.158 --port=9000
```

If the `host` or the `port` are not provided on the command line, and a MOOS file is also not provided, the user will be prompted for the two values. Since the most common scenario by convention has the MOOSDB running on the local machine ("localhost") with port 9000, these are the default values and the user can simply hit the return key.


```
$ uQueryDB --condition="DEPLOY=true" // User launches with no server host/port info
$ Enter Server: [localhost] // User accepts default by hitting Return key
$ The server is set to "localhost" // Server host confirmed to be set to "localhost"
$ Enter Port: [9000] 9123 // User overrides the default 9000 port with 9123
$ The port is set to "9123" // Server port confirmed to be set to "9123"
```

4.1 Logic Pass Conditions on the Command Line

On the command line **uQueryDB** takes any number of logic pass conditions. If *all* pass conditions are satisfied, then **uQueryDB** will return with 0, indicating success. A pass condition may be specified with either the `--condition` or `--pass_condition` parameter. They are interchangeable. A conjunction of pass conditions can be applied simply by specifying multiple conditions:

```
$ uQueryDB alpha.moos --condition="DEPLOY=true" --condition="STATION=false"
```

A disjunction of conditions can be specified as described in <http://oceanai.mit.edu/ivpman/logic>, within a single condition. For example:

```
$ uQueryDB alpha.moos --condition="((DB_TIME>36000) or (MISSION=complete))"
```

Since white-space characters on a command line delineate arguments, the use of double-quotes must be used if using white space as in the examples above.

4.2 Logic Fail Conditions on the Command Line

Similar to the pass conditions described above, a *fail* condition may also be specified. Depending on what you're trying to do, it may be just easier to express things with a fail condition(s) instead of pass conditions. If *any* fail condition is satisfied, then **uQueryDB** will return with a value of 1, indicating failure. For example:

```
$ uQueryDB alpha.moos --fail_condition="DEPLOY=false" \
--fail_condition="STATION=true"
```

Pass and fail conditions may both be used simultaneously. If *all* pass conditions are satisfied, and *no* fail conditions are satisfied, then **uQueryDB** will return with a value of 0, success. Otherwise it will return with a value of 1, failure.

4.3 Waiting for Success

A note of caution: by default, **uQueryDB** will immediately connect to the MOOSDB, evaluate conditions and return with either a 0 or 1. In some cases the user may want to allow **uQueryDB** a period of time before exiting with failure. The `--wait=N` command line parameter may be used, with the default being `N=0`. If **uQueryDB** detects success, it will return immediately. Otherwise, if the pass conditions are not initially all satisfied, or if one or more fail conditions are satisfied initially, **uQueryDB** will continue to evaluate for *N* seconds before returning with failing (1) value.

4.4 Waiting for a Logic Variable to be Published

Normally a logic condition of the form `(FOO<20)` would evaluate to false if the variable `FOO` has never been written to. This may seem counter-intuitive in conditions like `(DEPLOY!=true)` since clearly `DEPLOY` does not have the value of true if `DEPLOY` has never been written to. But nevertheless the latter condition would also evaluate to false.

5 Check Variables

`uQueryDB` can be optionally configured to report the values of one or more *check variables* at the time that `uQueryDB` exits, regardless of the return value. Check variables are named on either the command line or in the mission configuration. A variety of formats are supported. The motivation for check variables is to enable `uQueryDB` to be further useful in scripts that may be involved in automated mission testing.

5.1 Specifying Check Variables on the Command Line

One or more check variables may be specified on the command line. If variables are also specified in the mission file, the union of all variables will be used.

```
$ uQueryDB --check_var=RESULT --check_var=SCORE
```

5.2 Specifying Check Variables in the Mission File

One or more check variables may be specified in the mission file. If variables are also specified on the command line, the union of all variables will be used.

```
check_var = RESULT
check_var = SCORE
```

5.3 Check Variable Output and Format

The output of check variables is written to a file `.checkvars`. This file will be overwritten upon each run of `uQueryDB`. By default the output will be `equals-separated-value`. For example:

```
RESULT = pass
SCORE = 98
```

Another format option is `comma-separated-value`. For example:

```
RESULT,pass
SCORE,98
```

Another format option is `whitespace-separated-value`. For example:

```
RESULT pass
SCORE 98
```

Another format option is value-only. For example:

```
pass
98
```

The format option can be specified on the command line with `--esv` for equals-separated-value, or `--csv` for comma-separated-value, or `--wsv` for whitespace-separated-value, `--vo` for value-only.

The format option can also be specified in mission file with:

```
check_var_format = esv          // Default setting
check_var_format = wsv
check_var_format = csv
check_var_format = vo
```

If conflicting formats are specified between the command line and the mission file, the mission file setting will take precedence.

6 Configuration Parameters for uQueryDB

In addition to accepting query parameters from the command line, **uQueryDB** may accept configuration parameters from a configuration block with a `.moos` file, similar to most MOOS applications. This feature was added *after* Release 19.8.x.

The following parameters are defined for **uQueryDB**. A more detailed description is provided in other parts of this section. Parameters having default values indicate so in parentheses below.

Listing 6.1: Configuration Parameters for uQueryDB.

- check_var:** When the query results in a return value of zero, an optional variable report can be written to a file. Section 5.
- check_var_format:** Sets the output format for check variables written to the `.checkvars` file. Options are `esv`, or `csv`, or `wsv`, or `vo`. The default is `esv`. 5.3.
- condition:** Same as `pass_condition`. When *all* pass conditions are met, the query will result in a return value of zero. Section 4.1.
- fail_condition:** When *any* fail condition is unsatisfied, the query will result in a return value of zero. Section 4.2.
- pass_condition:** When *all* pass conditions are met, the query will result in a return value of zero. Section 4.1.
- wait:** If **uQueryDB** initially fails, rather than returning with a value of 1 immediately, it will wait *N* seconds to receive new mail updates that may perhaps result in successfully satisfying conditions. Default is 0 seconds. Section 4.3.

7 Publications and Subscriptions for uQueryDB

Variables published by the uQueryDB application

The **uQueryDB** application publishes no variables. In fact there will likely be no trace at all in a mission alog file, with the possible exception being the MOOS variable **DB_CLIENTS**. If **uQueryDB** does not immediately find the values for all variables involved in the logic expression, it will stay connected for as long as the `--wait` parameter specifies. The default is 0 seconds. In this case **uQueryDB** will be logged in the list of clients in **DB_CLIENTS** during this period.

Variables subscribed for by the uPokeDB application

All MOOS variables involved in the logic conditions provided to **uQueryDB** will be subscribed for automatically.